

Locks

A fundamental problem in concurrent programming.

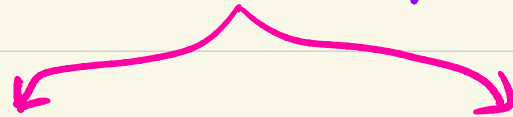
↳ to execute a series of instructions atomically.

↳ How? → using locks

An example

```
1 lock_t mutex; // some globally-allocated lock 'mutex'
2 ...
3 lock(&mutex);
4 balance = balance + 1; → critical section
5 unlock(&mutex);
```

The lock variable (such as mutex above) stores the state of the lock.



available (unlocked)

no thread has the lock currently

acquired (locked)

exactly one thread holds the lock.

* additional information (like, which thread is holding the lock) can also be included in the lock structure.

Two important routines:

① `lock()`: tries to acquire the lock

if lock is available → the calling thread acquires the lock

if some other thread is holding the lock → `lock()` will not return.

② `unlock()`: lock is now available

if no other threads are waiting for the lock → the lock is free

if there are waiting threads → one of them acquires the lock and enters the critical section.

An example of a POSIX code that performs similar things-

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3 pthread_mutex_lock(&lock); // wrapper; exits on failure
4 balance = balance + 1;
5 pthread_mutex_unlock(&lock);
```

Note that we can have multiple different locks protecting different resources, instead of a global lock → coarse-grained strategy

A coarse-grained strategy example

```
int balance = 0;           // shared global variable
FILE *logfile;             // shared global resource
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; // one global lock

void *worker(void *arg) {
    // --- Critical Section 1: update balance ---
    pthread_mutex_lock(&lock);
    balance += 10; // update shared data
    pthread_mutex_unlock(&lock);

    // --- Critical Section 2: write to shared logfile ---
    pthread_mutex_lock(&lock);
    fprintf(logfile, "Thread %ld updated balance to %d\n",
            pthread_self(), balance);
    pthread_mutex_unlock(&lock);

    return NULL;
}
```

A fine-grained strategy example

```
pthread_mutex_t balance_lock;
pthread_mutex_t log_lock;

pthread_mutex_lock(&balance_lock);
balance = balance + 1;
pthread_mutex_unlock(&balance_lock);

pthread_mutex_lock(&log_lock);
fprintf(logfile, "Balance updated\n");
pthread_mutex_unlock(&log_lock);
```

Question: How to build a lock?

Hardware
support?

OS support?

What are the parameters for evaluating the efficiency of locks?

① correctness

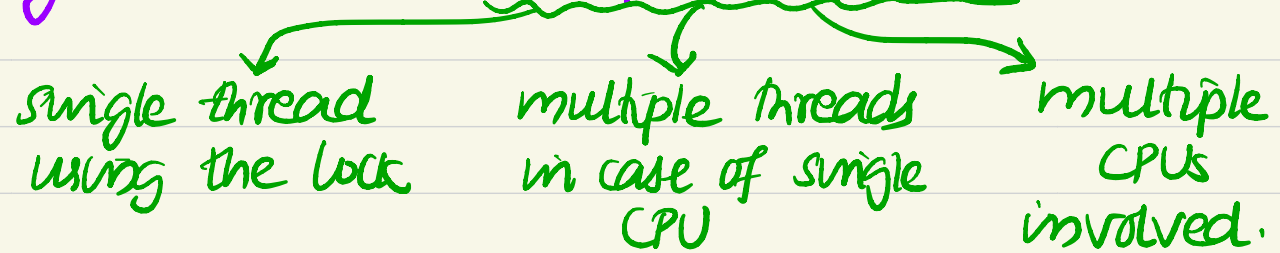
- mutual exclusion: only one thread at a time in the critical section.

② fairness

- starvation free: must eventually allow each waiting thread to enter.

(3) Performance

- time overheads for using the lock in multiple scenarios.



Attempt 0: Controlling interrupts

```
1 void lock() {  
2     DisableInterrupts();  
3 }  
4 void unlock() {  
5     EnableInterrupts();  
6 }
```

Idea: Disable interrupts in critical section $\Rightarrow$ no context switch can occur

$\Rightarrow$ no other thread/process can run concurrently

$\Rightarrow$ mutual exclusion achieved.

This technique originated in very early single CPU systems (like, early UNIX, MS-DOS, etc.)

Advantages:

* simplicity

* perfect mutual exclusion on single CPU.

Disadvantages:

* Dangerous and trust issues.

- disabling interrupts is a privileged instruction
- if a process forgets to re-enable interrupts (or crashes), the entire system may freeze.
- a process can intentionally monopolize the processor.
- a malicious program could call `lock()` and go to an endless loop

* does not work in multi-processors -

- disabling interrupts on one CPU does not stop other CPUs.

* performance loss

- no interrupts → can't handle devices, timers, or preemption.

Even today, this idea is used in very limited, short kernel code sections:

- eg. - disabling interrupts for a few microseconds while updating hardware interrupt tables, scheduler queues, etc.

But never used in user-space or large critical sections.

Attempt 1: Just using loads/stores.

a lock built only using regular memory reads and writes.

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> lock is available, 1 -> held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1) // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1;          // now SET it!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

Idea: * check $flag == 1$

if NO, the lock is free.

* set $flag = 1$ to acquire lock.

* Then later,
 $unlock()$ sets it back to 0.

Advantages:

* very simple

But what about correctness? whether the above implementation guarantees mutual exclusion? Unfortunately, NO!

The core problem is Race condition.

two threads can "see" the lock as free before either sets it.

Thread 1

```
call lock ()
while (flag == 1)
interrupt: switch to Thread 2
```

```
flag = 1; // set flag to 1 (too!)
```

Thread 2

```
call lock ()
while (flag == 1)
flag = 1;
interrupt: switch to Thread 1
```

Thread 1

```
Reads flag → sees 0
```

```
Sets flag = 1
```

Thread 2

```
Reads flag → sees 0
```

```
Sets flag = 1
```

Flag

```
0
```

```
0
```

```
1
```

This happened because, regular loads (mutex → flag == 1) and stores (mutex → flag = 1) are not atomic.

What about performance?

Spin waiting wastes time waiting for another thread to release a lock.

endlessly checks the value of the flag

(this waste is exceptionally high in uniprocessor → think, why?)

Question: can we modify the above approach of using only load and store to correctly build a lock (under certain assumptions)?

Yes, using Petersen's algorithm in early systems.

Homework: Read about it.

Attempt 2: Building working spin locks with Test-And-set

The problem with earlier approach → "check" and "set" are not atomic.

To overcome this problem, the hardware designers gave us a single atomic instruction called Test-and-set (sometimes called, atomic-exchange or xchg).

Definition in C →

```
int TestAndSet(int *old_ptr, int new) {  
    int old = *old_ptr;    // fetch old value at old_ptr  
    *old_ptr = new;        // store 'new' into old_ptr  
    return old;            // return the old value  
}
```

By atomicity, we mean

→ the CPU guarantees it happens as one uninterruptible operation.

∴, the test-and-set instruction, returns the old value pointed to by the old_ptr, and simultaneously updates said value to new.

⇒ it enables to test the old value while simultaneously setting the memory location to new value.

A simple spin-lock using Test-and-set.

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0: lock is available, 1: lock is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

What about correctness?

Case 1: first thread acquires the lock.

Thread A	lock → flag	Result
calls TestAndSet(&flag)	0	returns 0, sets flag=1
condition == 1 is false (in while)	1	A enters critical section.

lock is acquired by Thread A.

case_2: Another thread tries to lock while it's taken

Thread B	lock → flag	Result
calls TestAndSet(&flag)	1	returns 1, sets flag=1
condition == 1 is true and while loop continues	1	B spins (busy wait)

B keeps spinning until A calls **unlock()**

case_3: Releasing the lock

```
void unlock(lock_t *lock) {  
    lock->flag = 0;  
}
```

This makes flag=0, and B's
next **TestAndSet()** returns 0
→ it enters into critical section

Whether the above approach is correct?

YES, the atomicity of **TestAndSet()** ensures no two threads can both succeed at setting flag from 0→1 at the same time.

⇒ only a single thread is allowed to enter the critical section at a time ⇒ Finally, we have a correct lock!

What about fairness?

- Can we guarantee that a waiting thread will ever enter the critical section?

Unfortunately, not necessarily!

A thread spinning may spin forever, under contention.

What about performance?

case-1: threads competing for a lock on a single processor.

What could be a bad scenario? → There are N threads who want to acquire the lock, the thread holding the lock is preempted within a critical section.

→ The scheduler might run every other $N-1$ threads, each of them get stuck and spin during their whole duration of CPU cycle, resulting wastage of CPU time.

case-2: threads competing for a lock on multi-processors

spin locks might work reasonably well, if the #threads roughly equals the #CPUs.